

An Analysis of Gradient boosting

Gradient boosting

Gradient boosting -- Part 1

$$\gamma_m = \operatorname{argmin} \gamma \sum_{i=1}^n L(y_i, F_m(x_i)) = \operatorname{argmin} \gamma \sum_{i=1}^n L(y_i, F_{m-1}(x_i) - \gamma \nabla F_{m-1} L(y_i, F_{m-1}(x_i)))$$
$$\gamma_m = \operatorname{argmin} \gamma \sum_{i=1}^n L(y_i, F_{m-1}(x_i) - \gamma \nabla F_{m-1} L(y_i, F_{m-1}(x_i)))$$

Gradient boosting -- Part 2

$$F_m(x) = F_{m-1}(x) + \sum_{j=1}^J \gamma_j R_j(x), \gamma_j = \operatorname{argmin} \gamma \sum_{i \in R_j} L(y_i, F_{m-1}(x_i) + \gamma)$$
$$F_m(x) = F_{m-1}(x) + \sum_{j=1}^J \gamma_j R_j(x)$$

Gradient boosting -- Part 3

Tree size The number J of terminal nodes in the trees is a parameter which controls the maximum allowed level of interaction between variables in the model. With $J = 2$ (decision stumps), no interaction between variables is allowed. With $J = 3$ the model may include effects of the interaction between up to two variables, and so on. J can be adjusted for a data set at hand. Hastie et al. comment that typically $4 \leq J \leq 8$ work well for boosting and results are fairly insensitive to the choice of J in this range, $J = 2$ is insufficient for many applications, and $J > 10$ is unlikely to be required.

Gradient boosting -- Part 4

Gradient tree boosting Gradient boosting is typically used with decision trees (especially CARTs) of a fixed size as base learners. For this special case, Friedman proposes a modification to gradient boosting method which improves the quality of fit of each base learner. Generic gradient boosting at the m -th step would fit a decision tree $h_m(x)$ to pseudo-residuals. Let J_m be the number of its leaves. The tree partitions the input space into J_m disjoint regions $R_{1m}, \dots, R_{J_m m}$ and predicts a constant value in each region. Using the indicator notation, the output of $h_m(x)$ for input x can be written as the sum:

Gradient boosting -- Part 5

$$F_m(x) = F_{m-1}(x) - \gamma \sum_{i=1}^n \nabla F_{m-1} L(y_i, F_{m-1}(x_i))$$
$$F_m(x) = F_{m-1}(x) - \gamma \sum_{i=1}^n \nabla F_{m-1} L(y_i, F_{m-1}(x_i))$$

Gradient boosting -- Part 6

$$\gamma_m = \operatorname{argmin} \gamma \sum_{i=1}^n L(y_i, F_{m-1}(x_i) - \gamma \nabla F_{m-1} L(y_i, F_{m-1}(x_i)))$$
$$\gamma_m = \operatorname{argmin} \gamma \sum_{i=1}^n L(y_i, F_{m-1}(x_i) - \gamma \nabla F_{m-1} L(y_i, F_{m-1}(x_i)))$$

Gradient boosting -- Part 7

When the loss $L(\cdot, \cdot)$ is mean-squared error (MSE) the coefficients γ_j coincide with the coefficients of the tree-fitting procedure b_j .

Gradient boosting -- Part 8

where $\gamma > 0$. For small γ , this implies that $L(y_i, F_m(x_i)) \leq L(y_i, F_{m-1}(x_i))$.

Gradient boosting -- Part 9

$$-\frac{\partial L_{\text{MSE}}}{\partial F(x_i)} = 2(y_i - F(x_i)) = 2h_m(x_i)$$
$$-\frac{\partial L_{\text{MSE}}}{\partial F(x_i)} = 2(y_i - F(x_i)) = 2h_m(x_i)$$

Gradient boosting -- Part 10

where b_{jm} is the value predicted in the region R_{jm} . Then the coefficients b_{jm} are multiplied by some value γ_m , chosen using line search so as to minimize the loss function, and the model is updated as follows:

Gradient boosting -- Part 11

Regularization Fitting the training set too closely can lead to degradation of the model's generalization ability, that is, its performance on unseen examples. Several so-called regularization techniques reduce this overfitting effect by constraining the fitting procedure. One natural regularization parameter is the number of gradient boosting iterations M (i.e. the number of base models). Increasing M reduces the error on

An Analysis of Gradient boosting

Gradient boosting

training set, but increases risk of overfitting. An optimal value of M is often selected by monitoring prediction error on a separate validation data set. Another regularization parameter for tree boosting is tree depth. The higher this value the more likely the model will overfit the training data.

Gradient boosting -- Part 12

Usage Gradient boosting can be used in the field of learning to rank. The commercial web search engines Yahoo and Yandex use variants of gradient boosting in their machine-learned ranking engines. Gradient boosting is also utilized in High Energy Physics in data analysis. At the Large Hadron Collider (LHC), variants of gradient boosting Deep Neural Networks (DNN) were successful in reproducing the results of non-machine learning methods of analysis on datasets used to discover the Higgs boson. Gradient boosting decision tree was also applied in earth and geological studies – for example quality evaluation of sandstone reservoir.

Gradient boosting -- Part 13

In the discrete case however, i.e. when the set H $\{\displaystyle \mathcal{H}\}$ is finite, we choose the candidate function h closest to the gradient of L for which the coefficient γ may then be calculated with the aid of line search on the above equations. Note that this approach is a heuristic and therefore doesn't yield an exact solution to the given problem, but rather an approximation. In pseudocode, the generic gradient boosting method is:

Gradient boosting -- Part 14

$$F_0(x) = \arg \min_{h \in H} \sum_{i=1}^n L(y_i, h(x_i)),$$
$$\{ \underset{h \in \mathcal{H}}{\arg \min} \sum_{i=1}^n L(y_i, h(x_i)) \},$$